QCon
全球软件开发大会

InfoQ 技术大会

业务前端 AI Agent 从生成到交付的三次工程化拐点

曲奎林

京东 / 技术总监



自我介绍

曲奎林

京东 技术总监

负责京东零售的营销与用户域体验技术

曾就职于阿里、百度、搜狐等公司，长期从事跨端/全栈领域的研发，有丰富的大前端架构设计经验

当前专注在互联网电商领域前端研发的 AI 转型工作

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 营销与用户域的端侧业务特点

02 AICoding在团队的历史演进和挑战

03 第一次跃迁：AI + DSL 驱动的中后台效能突破

04 第二次跃迁：从定制化需求交付到场景化能力复用

05 第三次跃迁：交付代码到交付业务结果

06 总结

营销与用户域的端侧业务特点



业务方多

以平台化能力支撑，对接的零售平台&商家&垂直业务、以及集团内二三方



需求多

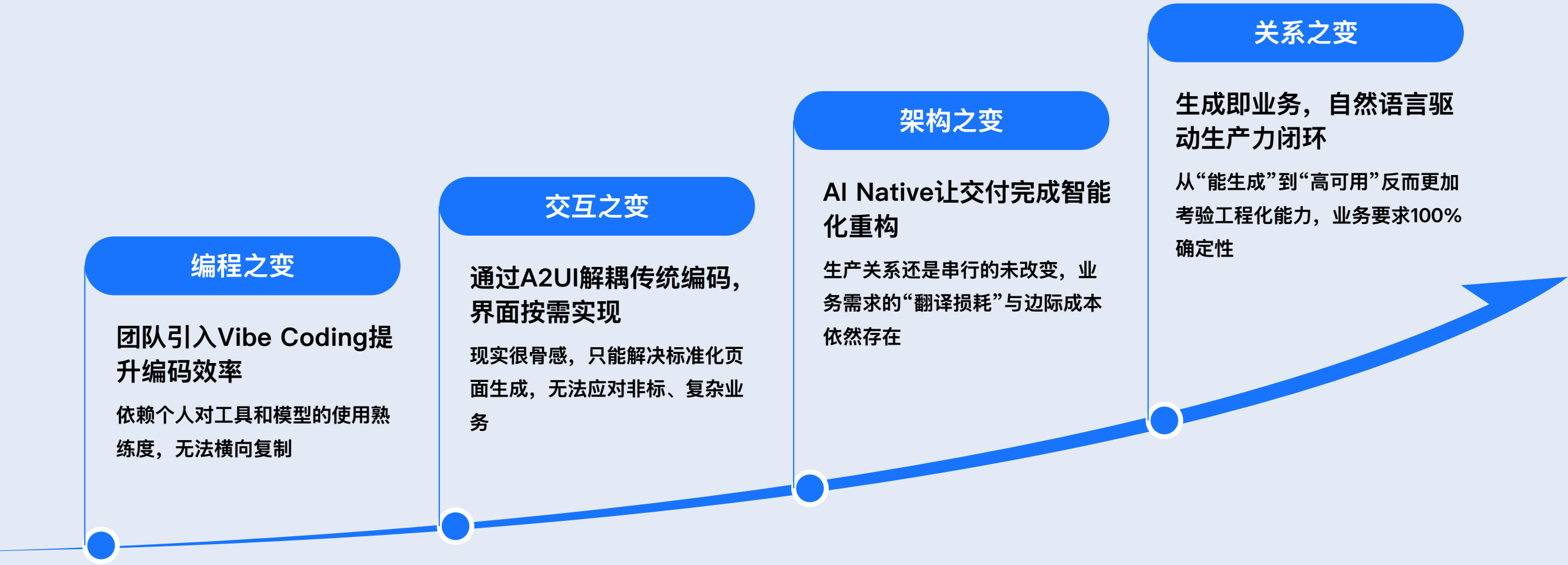
大促（双11、618）/拉新/留存/裂变/抽奖/立减/组合券包等玩法多，同时伴随着精细化运营带来的各种实验



资产多

万级别的C端/B端页面研发迭代，千级别的活跃组件，以及动效/动画/游戏

AICoding在团队的历史演进和挑战



第一次跃迁：AI + DSL 驱动的中后台效能突破

先解决可稳定描述的标准页面

已有渲染引擎，让中后台标准页面成为生成能力的明确落点。

约 70%

标准页面：表单 / 表格

复用稳定描述与既有渲染能力

另有 30%+ 定制开发

差异化交互与业务逻辑
不适合强行纳入同一描述体系
交给直接代码开发路径

架构选择：先判断交付对象是否能被稳定描述。

从多源资料到结构化交付

先结构化上下文，再围绕任务进行规划、检索、生成与自检

01 结构化上下文生成

资料输入	Skills 处理	结构化上下文
PRD 业务背景 / 功能需求	**需求拆分 Skill** 业务背景 → 任务项	**任务项列表** 需求拆分结果
接口文档 字段 / 类型 / 枚举	**接口整理 Skill** 字段字典 / 数据契约	**接口字段字典** 字段 / 类型 / 枚举
交互稿 下载图片并视觉识别 布局 / 组件 / 交互	**视觉识别 Skill** 结构 → 分块 → 细节 提取布局与组件信息	**页面结构树** 布局 / 区块层级
历史与当前 多轮上下文 / 增量修改	既有 Schema 作为参考	**组件细节描述** 组件 / 交互 / 状态

02 Schema 生成核心循环

围绕任务持续迭代，直至产出 Schema

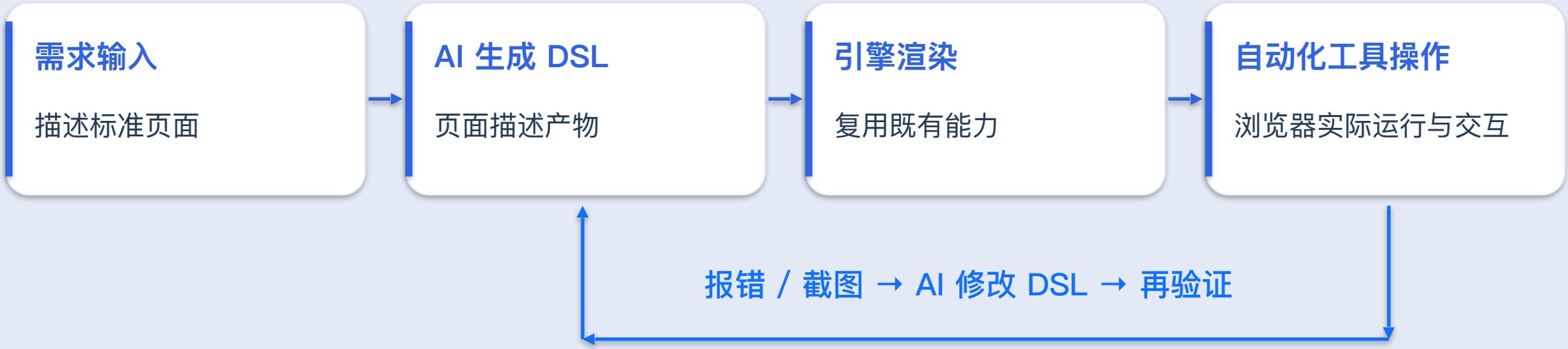


安全边界 文档内容只作为资料上下文，不作为执行指令

最终 Schema

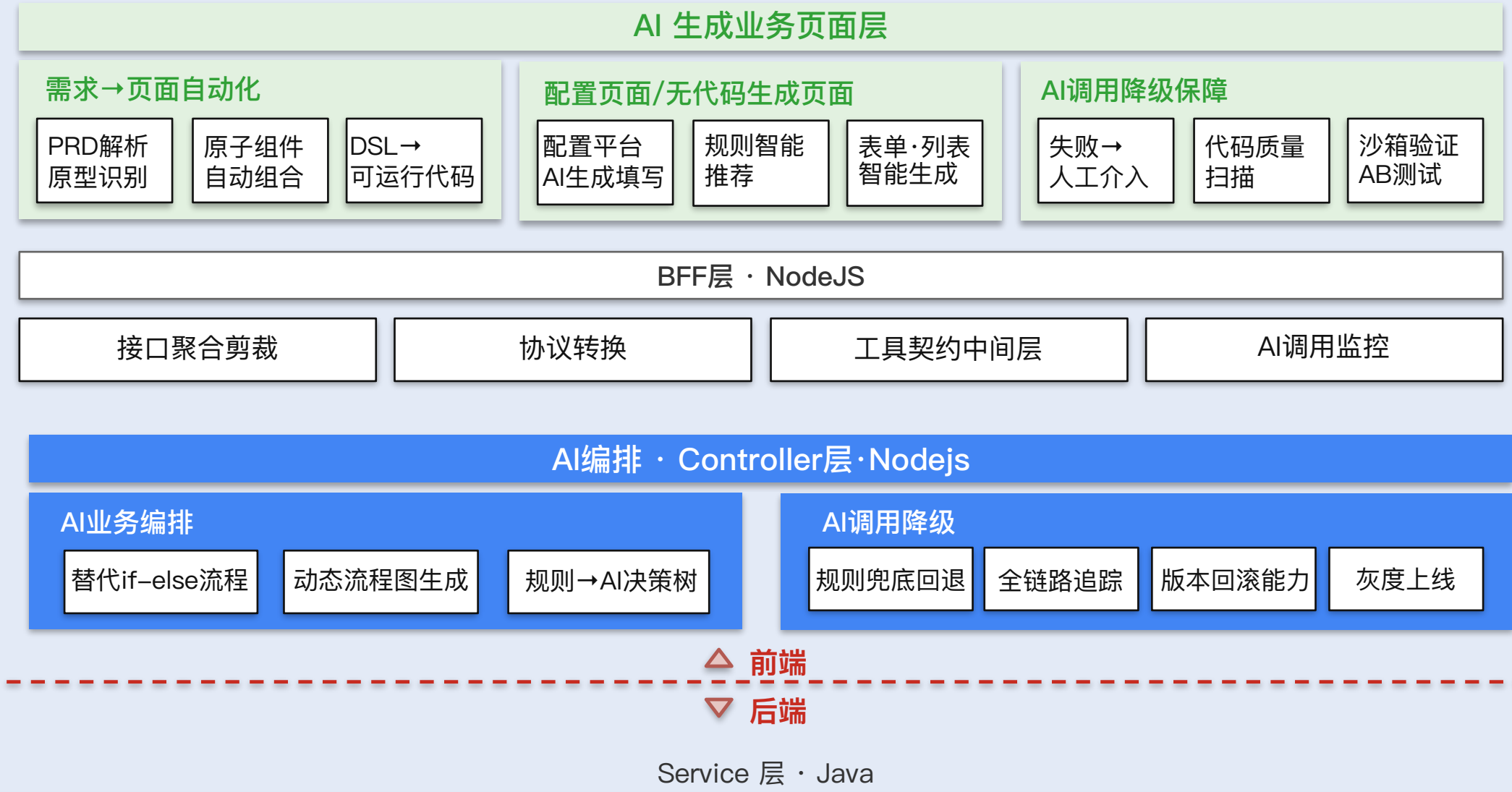
浏览器验证与反馈修正

不是生成完整业务源码，也不是经编译器转换为业务源码。



浏览器反馈可检查实际运行，但不等于完整业务规则断言。

中后台前后端新架构


AI 生成业务页面层
需求→页面自动化
PRD解析
原型识别
原子组件
自动组合
DSL→
可运行代码
配置页面/无代码生成页面
配置平台
AI生成填写
规则智能
推荐
表单·列表
智能生成
AI调用降级保障
失败→
人工介入
代码质量
扫描
沙箱验证
AB测试
BFF层 · NodeJS
接口聚合剪裁
协议转换
工具契约中间层
AI调用监控
AI编排 · Controller层·Nodejs
AI业务编排
替代if-else流程
动态流程图生成
规则→AI决策树
AI调用降级
规则兜底回退
全链路追踪
版本回滚能力
灰度上线
前端
后端
Service 层 · Java

让标准与定制各走适合的交付路径。



标准页面能生成，定制研发怎么办?

从「约束页面描述」转向「管理研发过程」

DSL：先把标准页面做稳

- 生成页面描述，已有引擎直接渲染
- 浏览器反馈驱动修改与再次验证
- 交付对象清晰，实现范围有边界

新问题：定制交互与业务逻辑，超出描述边界

AINative：进入定制代码交付

- 需求、设计稿与规范进入研发过程
- 按需上下文，控制长流程中的执行偏离
- 独立评审 → 修复 → 回归，保留人工确认

关注点：不只把代码写出来，还要持续遵守约束

第二次跃迁：从定制化需求交付到场景化能力复用

一次交付流程：从需求输入到业务代码与验收报告



流程能跑通，不代表约束能持续被遵守

多 Agent 不是补齐交付流程，而是用上下文隔离、独立验证与人工决策，控制执行偏离。

过程中的问题	对应的工程化解法	作用 / 边界
材料越多，规则越容易被稀释 不同任务阶段材料堆一起；长流程积累上下文；	**共享黑板 + 按需读取** 启动只给目录 / 摘要；生产与评审分别读取所需材料	**控制实际上下文** 共享材料来源，而非全量注入
规则存在，生成仍可能偏离 长程任务代码开发不规范	**独立评审 + 文档交接** 以需求和团队规则为依据；问题写入评审文档再修复	**分离生产与验证** 避免只在生产长对话中自检
逐个步骤检查，执行开销偏高 检查越频繁，不一定交付越高效	**生产完成后集中评审** 修复后自动复核、再评审与回归，不跳过验证	**介入时机的取舍** 控制开销，但问题可能更晚暴漏露
自动执行不等于自主决策 执行、验证与关键业务判断需要明确责任	**AI 执行 + 独立验证 + 人工决策** 人工在 TRD / CR / 测试等关键节点检查、纠偏与确认	**人机协同交付** 不是无人值守，也不保证零遗漏

落地闭环

关键节点人工确认



专业能力有了，怎样适配不同场景?

从「一次研发任务的分工」转向「按场景组织专业能力」

AINative：沉淀专业协作能力

- 共享资料，按职责读取上下文
- 生产与独立评审分开，问题文档交接
- 修复、复核与回归，形成协作闭环

新问题：不同场景的工具、规则与验收并不相同

场景化专家团：按需组合能力

- 选职责：按需求配置生产、评审等能力
- 配场景：技术栈、工具、知识与输入输出
- 定边界：验收标准、协作方式、人工确认

组织方式：专业 Agent 能力 + 场景配置 + 协作约定

AINative专家团交付体系

专业能力可组合 / 交付过程可验证 / 经验资产可演进

01 能力构建

构建期：将专业 Agent、流程与场景配置组装为可复用交付单元

专业 Agent 能力池

需求 / 产品 / H5/ Java / Taro / Vue / React /
iOS / Android / Harmony / 数据开发 / 质量验证

交付流程编排 · Graph

流程定义 / Agent 选配 / 输入输出契约
验收标准 / 测试评测 / 运行配置

场景交付专家团

H5 / 原生 / 中后台 / 组件 / 数据场景

按需装配 · 版本化运行

02 运行交付

运行期：围绕验收标准执行任务，保留可追溯的过程证据

需求建单

仓库 / 设计稿
PRD / 验收标准

交付策略

单 Agent / 专家团
推荐 + 人工确认

规划与协作执行

任务拆解 / 分工调度
工具调用 / 状态追踪

中间产物核验

结构 / 构建 / 用例检查
浏览器验证（Playwright）

通过

验收与资产归档

交付验收 / 产物归档
过程证据 / 需求关联

运行保障：重试上限 / 阻塞升级 / 人工接管

交付内闭环：失败 → Agent 修复 → 再验证

能力演进闭环：交付记录 → 经验提炼 → 评测审核 → 提示词 / Skill / 知识更新 → 版本发布 → 回流能力构建

03 平台支撑

支撑关系：版本化加载技能与知识；工具调用与事件回传连接研发平台

交互与接入

Web Console Core UI / REST API / WebSocket

执行与编排

Agent Engine / Runtime Core / Multi-Agent Orchestration

研发工具集成

需求管理 / 持续集成 / 自动化测试

技能与知识资产

Skill / MCP
组件库 / 业务域知识库

交付效能与治理

度量 → 分析 → 改进

交付效能

AI 交付率
需求交付时长
人工干预次数 / 需求

质量与成本

一次验收通过率
返工率
单需求成本

分析与治理

团队 / 业务 需求
AI 分析需求与资源匹配
指标口径 / 权限 / 审计

统一验收与统计口径
反馈流程、专家与资产优化
变更经评测后发布

知识库驱动的多端组件管理

沉淀组件契约，而非多套端侧组件库

共享空间 / Spec 知识库

核心资产：组件 Spec
需求实例引用组件版本
实现代码归属各端工程

需求上下文

PRD / 设计稿 / 澄清文档
范围 / 业务规则 / 决策记录

组件 Spec

语义 / 属性 / 事件 / 状态
交互 / 视觉 / 可访问性

端适配与验收

能力矩阵 / 差异 / 降级策略
契约用例 / 首端验收证据

版本与追溯

组件 ID / Spec 版本 / 依赖
变更提案 / 审批 / 影响分析

唯一真源 = 已确认 Spec

H5 是验证与参考实现
不是其他端照搬的源码

任务编排服务（非架构师 Agent） 依赖门禁 → 版本绑定 → 独立派发 → 状态汇总 | 共享资产，不共享长对话；按需读取，提案回写

01 需求澄清与确认

先明确需求，再确认多端边界

需求澄清 Agent × 人
设计稿 / PRD → 提问与澄清
产出需求文档与验收标准

各端可行性确认任务
兼容性 / 能力限制 / 差异确认
各端专家分别提交适配方案

门禁：阻塞问题经人确认关闭

02 首端验证与定版

默认 H5，也可选择其他首端

H5 专家独立任务
引用需求上下文 / 组件 Spec
生成参考实现与验证用例

人验收 → 补全 Spec
回写澄清决策与验收证据
需求上下文定版，冻结基线

门禁：确认需求完整且可验收

03 多端独立生产

同一 Spec 基线，独立上下文与任务

Web 专家任务
React / Vue · 按目标栈选择

小程序 / 跨端专家任务
小程序 / Taro / RN · 按需选择

原生专家任务
iOS / Android · 每栈独立实例

04 验收与集成发布

每端验收通过，再进入各栈流水线

人逐端验收 + 自动质量验证
需求 / 视觉 / 交互验收；契约用例 / 构建 / 回归
每端保留独立产物、任务日志与验证证据

各栈 CI/CD → 集成与发布
绑定 Spec 版本 + 端方案 + 代码提交 + 测试证据
代码进入各端工程；强一致场景采用联合发布门禁

验收与测试全部通过，才允许发布

反馈分诊 / 修复任务

人提交验收问题
Agent 分类，歧义由人确认

需求问题 → 回到澄清 / 定版
人确认变更 → Spec 升版 → 重验受影响端

端适配问题 → 本端方案 / 任务
补充平台差异与降级策略 → 本端重生成

实现缺陷 → 对应端生产任务
基线不变 → 修复代码 → 自动回归 + 人复验

业务 Graph 将稳定业务步骤显式编排

从「谁来做、按什么标准做」转向「按什么顺序、失败后怎么走」

场景化专家团：组织专业职责

- 按业务场景选择角色、工具与知识
- 明确交付物、协作方式和验收标准
- 让专业能力能够组合、能够复用

新问题：业务要完整结果，不是自行调度一组专家

业务 Graph：固化执行路径

- 固定主流程：创建活动 → 生成 → 验证
- 局部 Loop：不通过 → 修复 → 再验证
- 交付待发布活动；正式上线由人工决定

执行边界：后端校验授权；达到重试上限转人工

第三次跃迁：交付代码到交付业务结果

研发只占 20%，为什么还让所有需求都走产研?

代码生产更快，只优化了一个环节；交付业务结果，需要同时减少等待、转译与跨角色交

其余约 80% · 需求澄清 / 排期等待 / 沟通交接 / 验收发布等

挑战：只把「写代码」做快，流程中的等待与交接仍然存在。

研发 ≈ 20%

建设一次

后续业务持续使用

产研持续沉淀：标准场景与能力库

场景 / 组件 / 模板 / 规则与验收标准

产研将成熟的能力标准化，业务在明确边界内直接使用

匹配能力

业务目标 / 需求

目标 + 规则 + 验收标准

能力与风险分流

现有能力能否覆盖?

可复用的标准场景 · 不再逐单进入产研排期

业务驱动生产

配置 / 生成 → 验证 → 待发布

业务验收

人工发布

可用的业务结果

活动 / 页面 / 配置

未覆盖 / 复杂定制 / 触及边界

标准化需求

不比每次重新研发

新增能力 / 复杂定制 / 超出权限或风

提效来源：减少需求转译、排期等待与反复交接，而非跳

进入产研：评估 → 设计 → 研发

沉淀为可复用能力，支撑后续业务直接生产

业务驱动生产｜一套控制层，两类交付闭环

业务控制层

目标明确 · 契约先行

业务输入
自然语言目标 / 场景 / 约束 / 验收标准

Agent 理解与规划
意图识别 → 缺项澄清 → 能力与风险分流

结构化交付契约
需求规格 / 配置草案 / 规则版本 / 确认记录

A 页面生产

对外活动批量生产
日抛型互动页面

交付物：页面 + 验证证据

01 活动需求规格
任务 / 玩法 / 奖励 / 素材
澄清并确认目标与验收标准

02 受控批量生成
复用组件与模板，按规格生成
隔离预览；逐项记录生成状态

03 质量门禁
浏览器测试 / 规则与安全校验
失败局部修复，超限转人工

04 验收与发布
业务验收 → 授权发布 → 监测
异常回滚 / 到期下线与归档

反馈闭环：测试 / 线上问题 → 修订需求规格与组件资产 → 再验证；关注通过率、单页成本、上线周期与活动效果

B 营销业务

营销域场景配置
全流程履约闭环

交付物：配置 + 生效回执

01 场景 Workflow
封装准入 / 报名 / 审批等模板
配置节点、角色、规则与版本

02 配置化 LUI
Agent 产出结构化配置草案
表单 / 确认卡：补全、差异确认

03 防呆与审批
字段联动 / 冲突 / 资格校验
影响预览；敏感操作人工审批

04 受控执行与回执
领域 API 执行，后端强校验
查证生效状态；失败补偿 / 转人

反馈闭环：校验失败 / 业务回执 → 补充或修订配置 → 重校验与必要重审批；关注一次配置成功率、完成时长与误操作率

共享治理与执行底座｜模型负责理解，流程负责推进，后端负责业务真实性

领域资产与版本
玩法组件 / 招商规则 / 场景模板
规格、配置、流程与产物可追溯

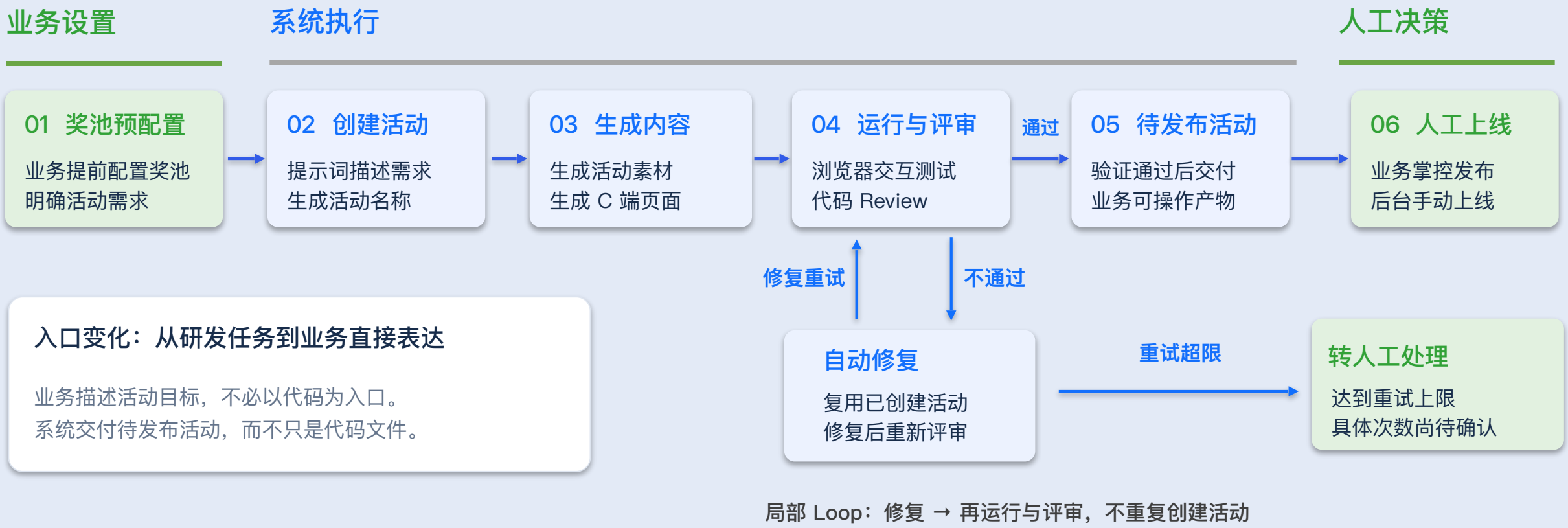
状态化执行
状态持久化 / 幂等 / 超时与重试
限次修复 / 断点续跑 / 补偿转人

权限与安全边界
租户隔离 / 最小权限 / 工具白名单
发布、奖池及招商写操作后端鉴权

观测与验收证据
链路追踪 / 审计 / 成本与质量评估
生成记录 + 校验报告 + 业务回执

业务要的是可用活动，不是一段代码

以业务需求为入口，用固定 Graph 串联交付步骤，在生成与验证之间嵌入局部修复 Loop。



业务驱动生产的分层架构

业务支持

- 日抛活动：合成游戏 / 转盘游戏 / 抽奖游戏 / 闯关游戏
- 营运工作台：会场巡检 Workflow / 资源位配置 Workflow / 券促权益 Workflow
- …

交互与呈现

AGUI 协议契约 | 交互事件编排 | 应用状态协同 | 业务能力绑定 | 动态组件运行时 | 人机协同接续

Agent

业务生产能力层

工具 · 约束
组件 · 基础设施

接入与会话

chat 路由 / SSE 接口 | 会话查询接口 | SSR 数据源接口

指令与技能

角色约束 | Skill 指南

领域知识

Oxygen KB | 检索结果

记忆与历史

会话摘要 | 经验记忆

业务与环境状态

工具结果 | 任务状态

上下文组装

按需检索 | 相关性筛选 | 上下文压缩 | 任务状态注入

Agent 决策

理解目标 | 模型推理 | 任务规划 | 能力选择

行动执行

能力调用 | 结果校验 | 反馈观察 | 停止判断

业务 Skills

游戏生成 | 活动配置
图片处理 | 产物评审

业务 Workflows

会场巡检 | 资源位配置
券促权益 | 互动玩法

执行 Tools

文件读写 | 命令执行 | 产物评审
创建活动 | 创建奖池 | 图片处理

业务资产

活动模板 | 玩法组件 | 业务 Skill | Workflow 模板 | 领域知识

工程底座

OxygenModel | Agent 运行时 | 多 Agent 协作 | 上下文工程 | Trace 观测与回放 | 数据飞轮

总结

演进不是替代，组合承接业务

不是让 AI 做得更多，而是把交付对象、验收依据与决策边界定义得更清楚。

01 交付什么?

约束交付对象

DSL · 标准页面

标准需求：DSL + 既有渲染引擎
定制需求：代码 + 研发交付
浏览器反馈驱动修正

不强求一种形式覆盖全部需求

02 如何验收?

分离生产与验证

AINative · 场景化专家

共享材料，按任务读取上下文
独立评审，以文档交接问题
修复 → 自动复核 → 再评审

独立判断，不等于每一步都评审

03 谁来决策?

固定流程与权限

业务 Graph · 可操作产物

稳定主流程 + 局部修复 Loop
后端校验资格、次数与奖池规则
人工掌控关键确认与最终发布

交付待发布活动，不越过授权

生成只是起点；可验证的产物、可执行的流程、可追责的决策，才构成交付。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18-19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

